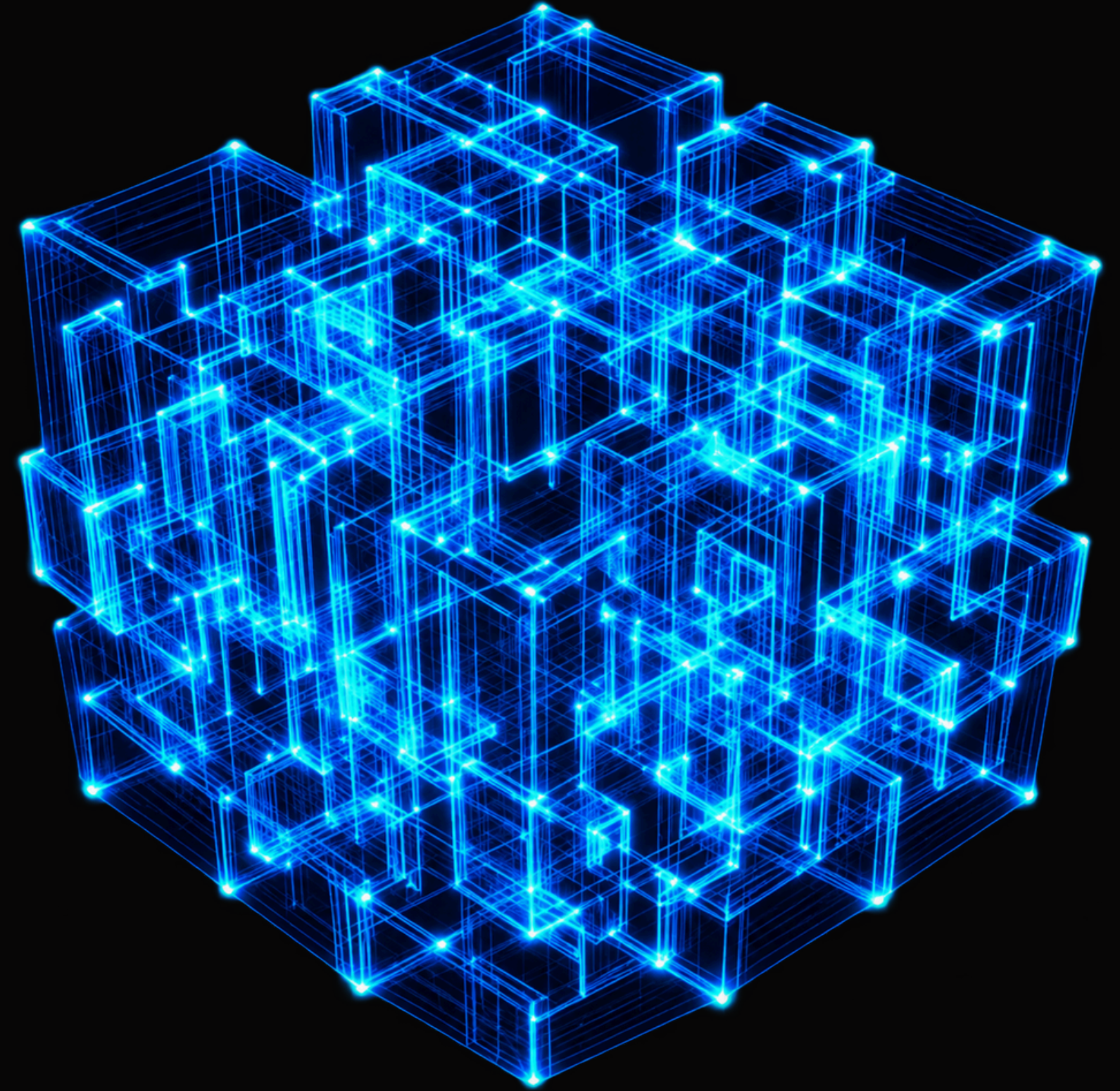


Matrix Chain Multiplication Optimization using Interval Dynamic Programming



Section – 7

Team Members:

2520030231 – K. Girish

2520030450 – P. Venkat

2520030307 – N.V.Karthikeya

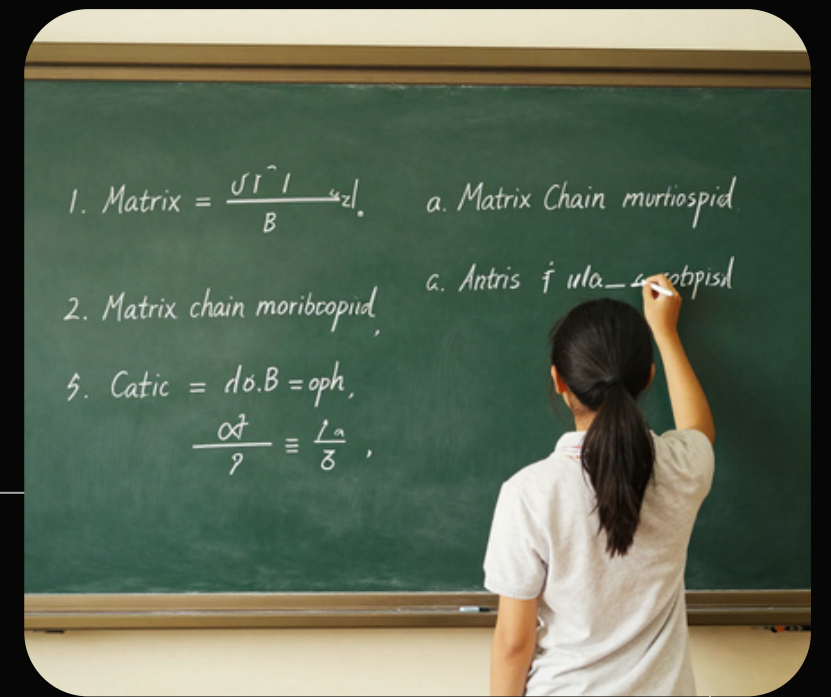
Team : 09

Abstract

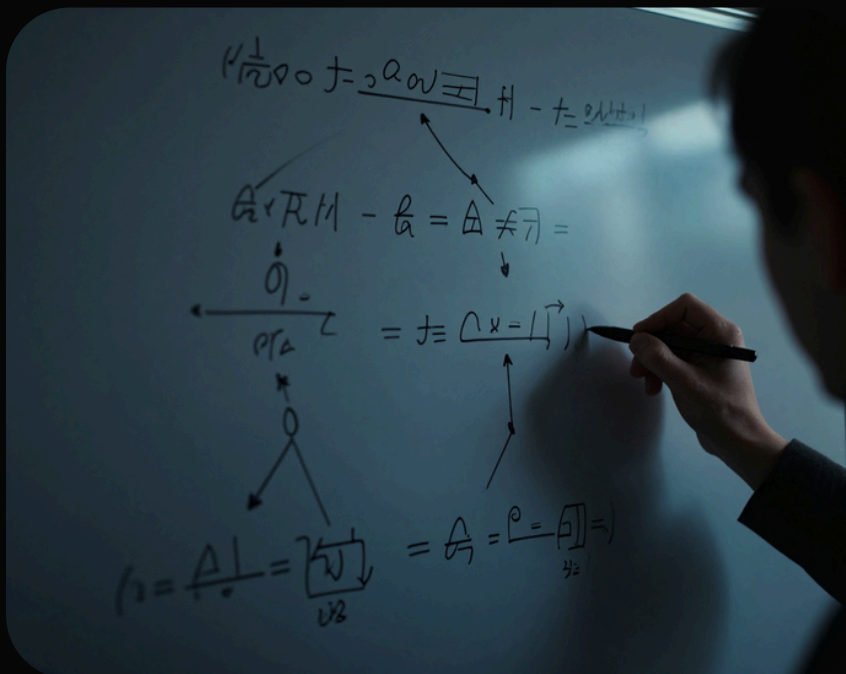
- **Matrix multiplication can be performed in different orders.**
- **Different orders can have different computational costs.**
- **Naïve multiplication may result in unnecessary operations.**
- **Interval Dynamic Programming is used to find the minimum-cost order.**
- **System displays:**
 - **Minimum multiplication cost**
 - **Optimal parenthesization**
 - **Naïve vs optimized cost**
 - **Efficiency gain**



Problem Statement

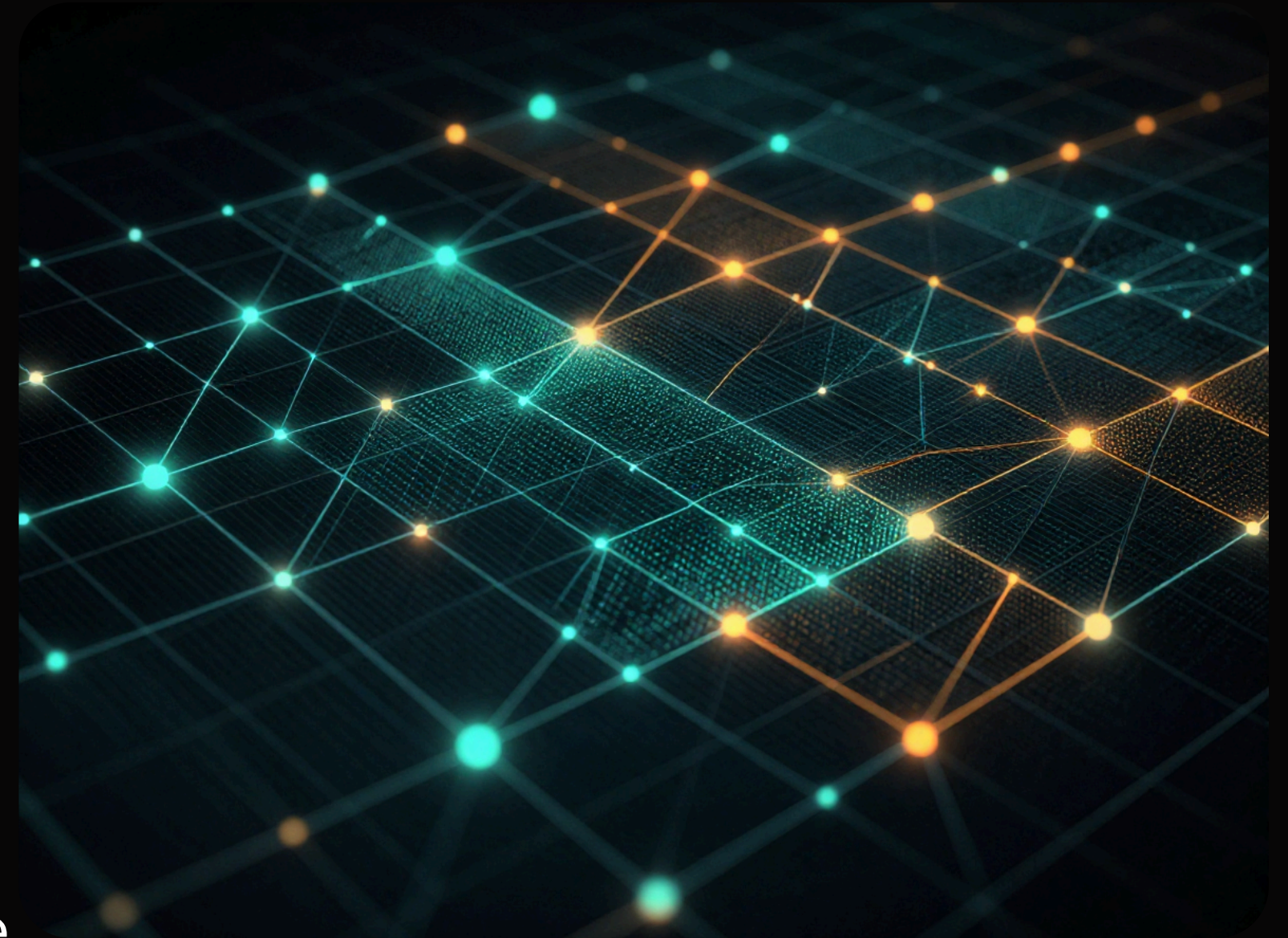


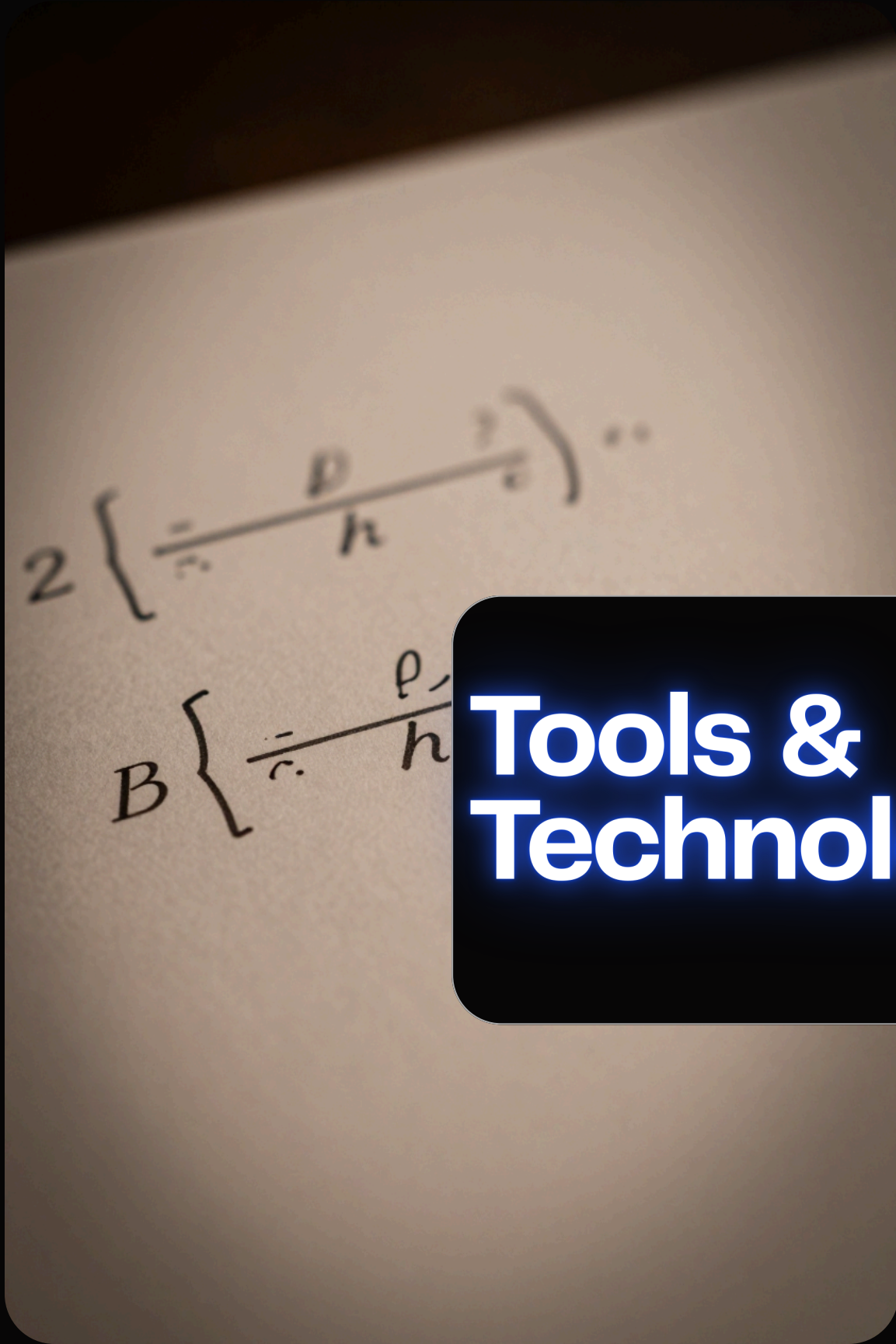
- Multiplication of multiple matrices has several possible orders.
- Each order may require a different number of scalar multiplications.
- Finding the best order manually becomes difficult for large matrix chains.
- A naïve left-to-right approach may be computationally expensive.
- Need an efficient optimization technique to:
 - Find the minimum multiplication cost
 - Determine the optimal multiplication order
 - Reduce unnecessary computations



Proposed Functionalities

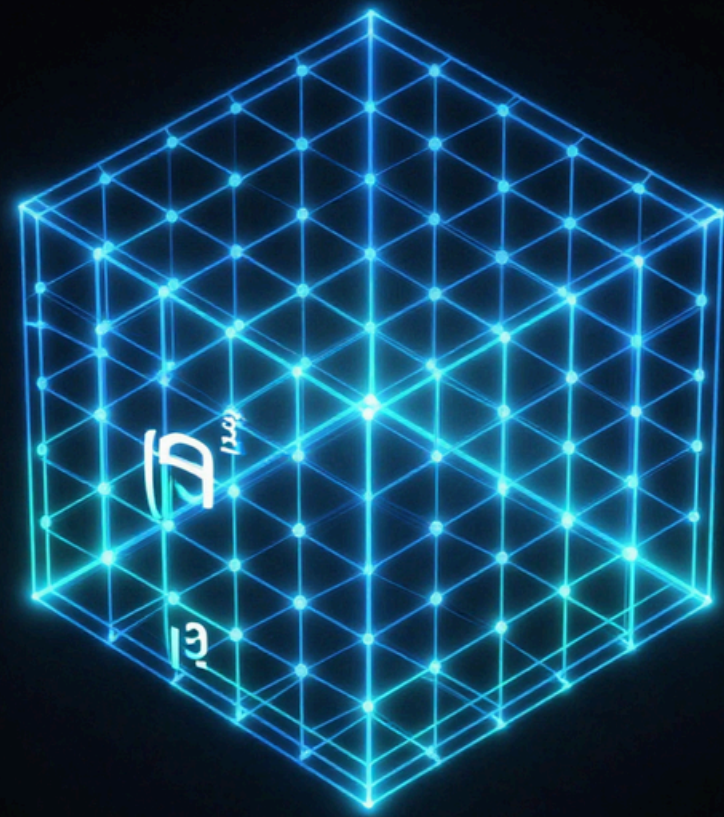
- Matrix InputEnter dimensions of matrices
- Validate compatible dimensions
- Interval DP OptimizationCalculate minimum multiplication cost
- Store results in a DP table
- Optimal ParenthesizationDetermine the best multiplication order
- Cost ComparisonCompare optimized and naïve approaches
- Efficiency AnalysisCalculate computational savings
- Report GenerationDisplay cost, order, and efficiency results





Tools & Technologies

- **Programming Language: Java**
- **Algorithm: Interval Dynamic Programming**
 - **Data Structures: 2D Arrays**
 - **DP Cost Table**
 - **Split Table**
 - **Development Environment: VS Code / IntelliJ IDEA**
- **Version Control: Git & GitHub**
- **Complexity:**
 - **Time: $O(n^3)$**
 - **Space: $O(n^2)$**



- Find the minimum scalar multiplication cost
- Generate the optimal parenthesization
- Reduce unnecessary computational operations
- Compare naïve vs optimized multiplication
- Calculate efficiency improvement
- Demonstrate practical use of Interval Dynamic Programming
- Provide clear and understandable optimization results

Expected Outcomes

Thank You

